

Tworzenie aplikacji webowych

Streamlit 2

STUDIA PODYPŁOMOWE

Data Science w zastosowaniach biznesowych

Uniwersytet Warszawski, Wydział Nauk Ekonomicznych



UNIwersYTET
WARSZAWSKI



WYDZIAŁ NAUK
EKONOMICZNYCH

Ewa Weychert, PhD

Copyright © 2025 WNE UW. All rights reserved.

Aplikacja do wczytywania danych i EDA — app_5.py

- Aplikacja zbudowana jest przy użyciu:
 - pandas do wczytywania i manipulowania danymi,
 - streamlit do tworzenia interfejsu webowego,
 - matplotlib i seaborn do wizualizacji.
- Główny tytuł `st.title("Data Loading & EDA App")` pojawia się na górze strony.
- Widget **przesyłania pliku** pozwala użytkownikowi wgrać plik CSV:
 - wgrany plik jest przechowywany w zmiennej `uploaded`,
 - jeśli plik zostanie dostarczony, jest wczytywany do `DataFrame` za pomocą `pd.read_csv`.
- Wyświetlany jest podgląd zestawu danych (pierwsze 5 wierszy), aby dać szybki wgląd w dane.

Funkcje pomocnicze dla EDA — app_5.py

- **Funkcja podstawowych informacji**

- Wyświetla podnagłówek „Podstawowe informacje”.
- Pokazuje kształt DataFrame (liczbę wierszy i kolumn).
- Wyświetla typy danych każdej kolumny.
- Drukuje podstawowe statystyki opisowe dla zmiennych numerycznych.

- **Funkcja brakujących wartości**

- Wyświetla podnagłówek „Brakujące wartości”.
- Dla każdej kolumny pokazuje, ile brakujących wartości zawiera.

- **Funkcja uzupełniania braków średnią**

- Oblicza średnią wszystkich kolumn numerycznych.
- Zwraca wersję DataFrame, w której numeryczne wartości NaN są zastąpione odpowiednią średnią kolumny.

- **Funkcja rozkładów kategoriycznych**

- Identyfikuje wszystkie kolumny kategoriyczne (typu object).
- Dla każdej kolumny kategoriycznej drukuje liczebności wartości.
- Używa linii separatora, aby ułatwić czytanie wyniku.

Interaktywne opcje w aplikacji — app_5.py

- Po wczytaniu danych użytkownik może wybrać dodatkowe działania za pomocą checkboxów:
 - **Kodowanie one-hot**
 - Jeśli użytkownik zaznacza checkbox, wszystkie kolumny katagoryczne są konwertowane na zmienne dummy przy użyciu kodowania one-hot.
 - Komunikat sukcesu potwierdza zastosowanie `pd.get_dummies()`.
 - **Pokaż liczebności wartości katagorycznych**
 - Jeśli wybrane, aplikacja wywołuje funkcję, która drukuje liczebności wartości dla wszystkich zmiennych katagorycznych.
- Dzięki temu aplikacja jest elastyczna: użytkownik decyduje, czy transformować dane i czy sprawdzać rozkłady katagoryczne.

Wizualna eksploracyjna analiza danych — app_5.py

- Osobna sekcja „Wizualna eksploracyjna analiza danych” jest poświęcona wykresom.
- Aplikacja najpierw wyodrębnia wszystkie kolumny numeryczne:
 - jeśli nie ma kolumn numerycznych, wyświetlane jest ostrzeżenie i żadne wykresy nie są tworzone.
- Jeśli dostępne są dane numeryczne, użytkownik może:
 - wybrać zmienną numeryczną do **analizy rozkładu**:
 - tworzony jest histogram z krzywą gęstości (KDE),
 - wyświetlany jest również wykres pudełkowy dla tej samej zmiennej.
 - obejrzeć **macierz korelacji**:
 - obliczane są współczynniki korelacji między zmiennymi numerycznymi,
 - wyświetlane jako mapa ciepła ze skalą kolorów.
 - utworzyć **wykres rozrzutu** między dwiema zmiennymi numerycznymi:
 - użytkownik wybiera zmienną dla osi X i zmienną dla osi Y,
 - rysowany jest wykres rozrzutu pokazujący ich zależność.
- Wszystkie wykresy są renderowane przy użyciu seaborn i wyświetlane w Streamlit za pomocą st.pyplot.

Wczytywanie danych i EDA — app_5.py

Data Loading & EDA App

Upload your CSV

 Drag and drop file here
Limit 200MB per file

Browse files

 cleaned_data.csv 112.6KB

X

Preview

passenger_id	pclass	survived	name	sex
0	1	1	Allen, Miss. Elisabeth Walton	female
1	2	1	Allen, Master. Hudson Trevor	male
2	3	0	Allson, Miss. Helen Louise	female
3	4	1	Allson, Mr. Hudson Joshua Creighton	male
4	5	0	Allson, Mrs. Hudson J.C. (Bessie Waldo Daniels)	female

☒ Apply one-hot encoding to categorical columns

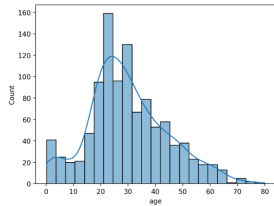
Applied `pd.get_dummies()`

☐ Show categorical value counts

Visual Exploratory Data Analysis

Choose numeric column (distribution)

age



Prosta aplikacja EDA — app_6.py

- Ten przykład demonstruje bardzo małą aplikację Streamlit do **podstawowej eksploracyjnej analizy danych**.
- Aplikacja używa dwóch bibliotek:
 - `pandas` do wczytywania i inspekcji danych tabelarycznych,
 - `streamlit` do budowania interaktywnego interfejsu.
- Użytkownik wgrywa plik CSV, który następnie jest wczytywany do `DataFrame` `pandas`.
- Wykonywane są dwie główne analizy:
 - 1 wyświetlanie ogólnych informacji o zestawie danych,
 - 2 pokazywanie rozkładów zmiennych kategorycznych.

Funkcja: Podstawowe informacje — app_6.py

- Pierwsza funkcja pomocnicza odpowiada za drukowanie prostych informacji strukturalnych o zestawie danych.
- Wyświetla nagłówki sekcji zatytułowany „*Podstawowe informacje*”.
- Funkcja pokazuje:
 - **kształt** zestawu danych (tj. liczbę wierszy i kolumn),
 - **typy danych** każdej kolumny (numeryczne, tekstowe itd.).
- Ten krok pomaga użytkownikom szybko zrozumieć, jak duży jest zestaw danych i jakie typy zmiennych zawiera.

Funkcja: Rozkłady kategoriyczne — app_6.py

- Druga funkcja pomocnicza dostarcza prostego podsumowania wszystkich **zmiennych kategoriycznych**.
- Identyfikuje kolumny typu object, które zazwyczaj reprezentują kategorie lub tekst.
- Dla każdej kolumny kategoriycznej:
 - drukowana jest nazwa kolumny,
 - pokazywana jest częstość (liczba) każdej kategorii.
- Zaimplementowane jest to za pomocą **pętli**, która automatycznie przetwarza wszystkie kolumny kategoriyczne.
- Daje szybki przegląd rozkładu kategorii — przydatny do EDA i czyszczenia danych.

Wczytywanie danych i przepływ wykonania — app_6.py

- Aplikacja zawiera widget **przesyłania pliku**, który pozwala użytkownikowi wgrać plik CSV.
- Jeśli plik zostanie wgrany:
 - 1 plik jest wczytywany do DataFrame,
 - 2 wykonywana jest funkcja **podstawowych informacji**,
 - 3 wykonywana jest funkcja **rozkładów kategoriycznych**.
- Dzięki temu aplikacja jest reaktywna:
 - gdy tylko użytkownik dostarczy zestaw danych, podsumowania EDA pojawiają się automatycznie.
- Ta prosta struktura demonstruje podstawowy wzorzec aplikacji Streamlit: **wgraj → wczytaj → analizuj → wyświetl**.

Wczytywanie danych i EDA — app_6.py

Upload your CSV

Drag and drop file here
Limit 200MB per file

Browse files

cleaned_data.csv 112 KB X

Basic Info

Shape: (1309, 15)

passenger_id	int64
pclass	int64
survived	int64
name	object
sex	object
age	float64
sibsp	int64
parth	int64
ticket	object
fare	float64

Categorical Distributions

name

name	count
Crossley, Miss. Kate	2
Kelly, Mr. James	2
Allen, Miss. Elisabeth Walton	1
Brenkanger, Miss. Ida Looja	1
Staff, Mr. Flu	1
Shahen Shahseh, Mr. Yussef	1
Hymen, Mr. Abraham	1
Humblen, Mr. Adolf Mathias Nicolas Olsen	1
Howard, Miss. May Elizabeth	1
Horgan, Mr. John	1

sex

sex	count
male	843
female	466

Organizowanie aplikacji za pomocą paska bocznego

Rola paska bocznego

- Pomaga organizować zawartość dashboardu.
- Poprawia doświadczenie użytkownika i strukturę.
- Często zawiera branding (logo, nazwę aplikacji) oraz nawigację lub filtry.

Typowe opcje układu

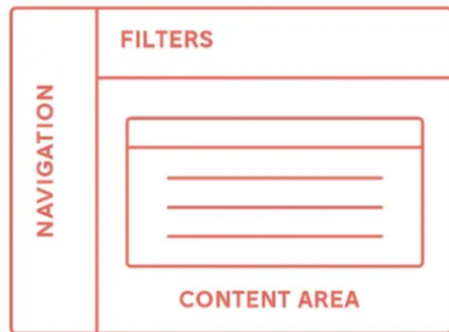
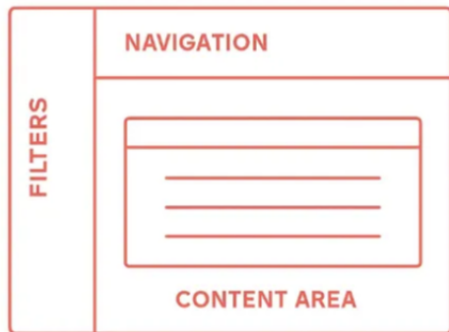
❶ Nawigacja na górze + filtry w pasku bocznym

- Menu nawigacyjne pojawia się jako górny pasek.
- Filtry i kontrolki są umieszczone w pasku bocznym.
- Najlepszy dla dashboardów z wieloma filtrami, ale niewielu sekcjami (pięć lub mniej).

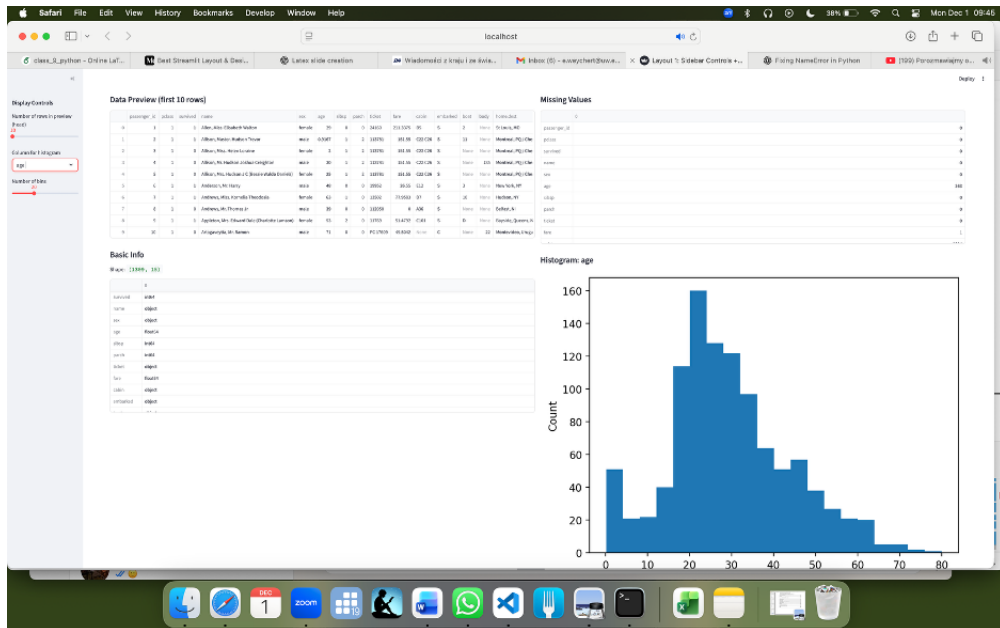
❷ Nawigacja w pasku bocznym + filtry na górze

- Menu nawigacyjne umieszczone jest w pasku bocznym.
- Filtry pojawiają się na górnym pasku.
- Idealne dla dashboardów z wieloma stronami/sekcjami i mniejszą liczbą filtrów.
- Częściej stosowane w projektowaniu dashboardów.

Porównanie typowych układów dashboardu z paskami bocznymi



Przykład z jedną zakładką — app_7.py



Ogólny przegląd — app_7.py

- Skrypt buduje interaktywny **eksplorator danych** przy użyciu Streamlit.
- Użytkownik wgrywa plik CSV za pomocą `st.file_uploader`.
- **Pasek boczny** kontroluje:
 - liczbę wierszy wyświetlanych w podglądzie,
 - liczbę przedziałów dla histogramów,
 - liczbę najpopularniejszych kategorii dla wykresów słupkowych.
- Główny obszar jest podzielony na **cztery zakładki**:
 - 1 Przegląd
 - 2 Braki i typy
 - 3 Zmienne ciągłe
 - 4 Zmienne kategoryczne

Konfiguracja strony i wczytywanie pliku — app_7.py

- `st.set_page_config` ustawia:
 - tytuł strony ("Data Explorer"),
 - układ ("wide").
- Główny tytuł i podpis są tworzone za pomocą:
 - `st.title("Data Explorer")`,
 - `st.caption(...)` do krótkiego opisu aplikacji.
- Użytkownik dostarcza dane poprzez:
 - `uploaded = st.file_uploader("Upload your CSV file")`.
- Po wgraniu pliku jest on wczytywany do DataFrame pandas:
 - `df = pd.read_csv(uploaded)`.

Kontrolki paska bocznego — app_7.py

- Dostęp do paska bocznego uzyskuje się przez `st.sidebar`.
- Trzy główne kontrolki:
 - ❶ **Wiersze w podglądzie:**
 - Suwak kontrolujący liczbę wierszy wyświetlanych w `df.head()`.
 - ❷ **Przedziały dla histogramów ciągłych:**
 - Suwak kontrolujący parametr `bins` w histogramach `matplotlib`.
 - ❸ **Top N kategorii:**
 - Suwak decydujący, ile najczęstszych kategorii pokazać na wykresach słupkowych.
- Wartości te są przekazywane do funkcji pomocniczych, aby zachować porządek w logice.

Zakładki i funkcje pomocnicze — app_7.py

- Aplikacja używa **zakładek** do strukturyzacji:
 - `tab_overview, tab_missing, tab_continuous, tab_categorical = st.tabs([...])`
- Każda zakładka wywołuje osobną funkcję pomocniczą:
 - `panel_overview(df, n_head)`
 - `panel_missing_and_types(df)`
 - `panel_continuous(df, bins)`
 - `panel_categorical(df, top_n)`
- Dzięki temu główny skrypt jest przejrzysty, a każdy panel łatwiejszy w utrzymaniu.

Zakładka Przegląd: 4 sekcje — app_7.py

- Używa dwukrotnie `st.columns(2)` do stworzenia siatki 2×2 .
- Cztery sekcje:
 - ❶ **Kształt i podgląd**
 - Pokazuje liczbę wierszy/kolumn i `df.head(n_head)`.
 - ❷ **Nazwy kolumn**
 - Wyświetla listę wszystkich nazw kolumn.
 - ❸ **Podsumowanie numeryczne**
 - `df.select_dtypes(include="number").describe()`.
 - ❹ **Podsumowanie kategoriyczne**
 - `df.select_dtypes(exclude="number").describe(include="all")`.

Zakładka Braki i typy: 4 sekcje — app_7.py

- Również podzielona na 4 sekcje przy użyciu dwóch rzędów po dwie kolumny:

- 1 **Liczba braków** dla każdej kolumny:

```
missing_count = df.isna().sum()
```

- 2 **Procent braków** dla każdej kolumny:

```
missing__ = df.isna().mean() × 100
```

- 3 **Tabela typów danych:**

```
dtypes = df.dtypes
```

- 4 **Liczba unikalnych wartości:**

```
n_unique = df.nunique()
```

- Każdy wynik jest wyświetlany jako mały DataFrame.

Zakładka Zmienne ciągłe: histogramy — app_7.py

- Wybiera wszystkie kolumny numeryczne:

```
num_cols = df.select_dtypes(include = "number").columns
```

- Dla każdej kolumny numerycznej:
 - Usuń brakujące wartości: `df[col].dropna()`.
 - Narysuj histogram przy użyciu `matplotlib`:
 - `fig, ax = plt.subplots()`
 - `ax.hist(data, bins=bins)`
 - `st.pyplot(fig)` do wyświetlenia w Streamlit.
- Wykresy są rozmieszczone w dwóch kolumnach Streamlit tworząc siatkę.

Zakładka Zmienne kategoryczne: wykresy słupkowe — app_7.py

- Wybiera wszystkie kolumny nienumeryczne (kategoryczne):

```
cat_cols = df.select_dtypes(exclude = "number").columns
```

- Dla każdej kolumny kategorycznej:

- Oblicz liczby częstości:

```
vc = df[col].value_counts().head(top_n)
```

- Narysuj wykres słupkowy przy użyciu matplotlib:

- `ax.bar(vc.index.astype(str), vc.values)`
- Obrót etykiet osi X dla czytelności.
- Wyświetlenie z `st.pyplot(fig)`.

- Ponownie używane są dwie kolumny Streamlit do stworzenia siatki wykresów.

Kluczowe idee — app_7.py

- **Rozdzielenie odpowiedzialności:**
 - Główny skrypt obsługuje układ i routing.
 - Funkcje pomocnicze obsługują zawartość każdej zakładki.
- **Interaktywne kontrolki** w pasku bocznym wpływają na wiele wizualizacji.
- **Spójny układ 4-sekcyjny** w każdej zakładce:
 - Używa `st.columns(2)` do budowania siatki 2×2 .
- **Matplotlib** jest używany do wszystkich wykresów, osadzanych w Streamlit za pomocą `st.pyplot`.

Czym różnią się app_7 i app_8

- Układ i nawigacja

- app_7: jedna **strona** z siatką 2×2 (podgląd, podstawowe informacje, brakujące wartości, jeden histogram).
- app_8: używa **wielu zakładek** (Przegląd, Braki i typy, Zmienne ciągłe, Zmienne kategoryczne), każda zakładka podzielona wewnętrznie na 4 sekcje.

- Zakres EDA

- app_7: podstawowy dashboard skupiony na szybkim przeglądzie i **jednej wybranej kolumnie numerycznej** dla histogramu.
- app_8: bardziej kompletny **eksplorator danych**: podsumowania dla zmiennych numerycznych i kategorycznych, tabele brakujących danych, typy danych, liczby unikalnych wartości i wiele wykresów.

- Wizualizacje

- app_7: pojedynczy histogram wybrany w pasku bocznym.
- app_8: histogramy dla **wszystkich** kolumn numerycznych i wykresy słupkowe dla **wszystkich** kolumn kategorycznych (ułożone w siatkach 2-kolumnowych).

- Przypadek użycia

- app_7: prosty szablon do pokazania układu *jednej strony* z 4 panelami.
- app_8: szablon dla *pełnej aplikacji EDA* ze strukturyzowaną nawigacją i bogatszą analizą.

Przykład z wieloma zakładkami — app_8.py

Display Controls

Rows: 1 (page 1 of 1)

Number of Rows (per column): 10

Apply Categories (categorical): 10

Data Explorer

Each table is loaded into four sections: overview, relationships, data types, and distributions for each feature and categorical variables.

Upload your CSV file

Drop and drag a file here

Local: 2.0 MB, per 5 MB

Dataset: titanic.csv (112,000)

Download File

Overview

Shape & Basic Info

Number of rows: 1558

Number of columns: 15

First 10 rows

passenger_id	status	survived	name	sex	age	ship	port	class	date	date1	embarked	boat	body	normalized
0	0	1	Allen, Miss Elsiebeth Walton	female	29	0	0	20019	10.0175	01	0	3	female	Staten, 100
1	2	0	Allen, Mr. Thomas Francis	male	35.00	1	2	01290	10.0175	00	0	10	female	Harland, 101/100
2	3	0	Allen, Mrs. Thomas Francis	female	2	2	2	01290	10.0175	00	0	100	female	Harland, 101/100
3	4	0	Allen, Mr. Thomas Francis	male	30	1	2	01290	10.0175	00	0	100	female	Harland, 101/100
4	5	0	Allen, Mrs. Thomas Francis	female	20	1	2	01290	10.0175	00	0	100	female	Harland, 101/100
5	6	0	Anderson, Mr. Harry	male	44	0	0	00010	10.0175	00	0	3	female	New York, 101
6	7	0	Anderson, Mrs. Harry	female	40	0	0	00010	10.0175	00	0	10	female	Harland, 101
7	8	0	Anderson, Mr. Thomas	male	30	0	0	00010	10.0175	00	0	100	female	Harland, 101
8	9	0	Anderson, Mrs. Thomas	female	30	0	0	00010	10.0175	00	0	100	female	Harland, 101
9	10	0	Anderson, Mr. Thomas	male	30	0	0	00010	10.0175	00	0	100	female	Harland, 101

Column Names

```
1 ["passenger_id",  
2 "status",  
3 "survived",  
4 "name",  
5 "sex",  
6 "age",  
7 "ship",  
8 "port",  
9 "class",  
10 "date",  
11 "date1",  
12 "embarked",  
13 "boat",  
14 "body",  
15 "normalized"]
```

Numeric Summary (describe)

passenger_id	status	survived	name	sex	age	ship	port	class	date	date1	embarked	boat	body	normalized
0	0	1	Allen, Miss Elsiebeth Walton	female	29	0	0	20019	10.0175	01	0	3	female	Staten, 100
1	2	0	Allen, Mr. Thomas Francis	male	35.00	1	2	01290	10.0175	00	0	10	female	Harland, 101/100
2	3	0	Allen, Mrs. Thomas Francis	female	2	2	2	01290	10.0175	00	0	100	female	Harland, 101/100
3	4	0	Allen, Mr. Thomas Francis	male	30	1	2	01290	10.0175	00	0	100	female	Harland, 101/100
4	5	0	Allen, Mrs. Thomas Francis	female	20	1	2	01290	10.0175	00	0	100	female	Harland, 101/100
5	6	0	Anderson, Mr. Harry	male	44	0	0	00010	10.0175	00	0	3	female	New York, 101
6	7	0	Anderson, Mrs. Harry	female	40	0	0	00010	10.0175	00	0	10	female	Harland, 101
7	8	0	Anderson, Mr. Thomas	male	30	0	0	00010	10.0175	00	0	100	female	Harland, 101
8	9	0	Anderson, Mrs. Thomas	female	30	0	0	00010	10.0175	00	0	100	female	Harland, 101
9	10	0	Anderson, Mr. Thomas	male	30	0	0	00010	10.0175	00	0	100	female	Harland, 101

Categorical Summary (describe)

passenger_id	status	survived	name	sex	age	ship	port	class	date	date1	embarked	boat	body	normalized
0	0	1	Allen, Miss Elsiebeth Walton	female	29	0	0	20019	10.0175	01	0	3	female	Staten, 100
1	2	0	Allen, Mr. Thomas Francis	male	35.00	1	2	01290	10.0175	00	0	10	female	Harland, 101/100
2	3	0	Allen, Mrs. Thomas Francis	female	2	2	2	01290	10.0175	00	0	100	female	Harland, 101/100
3	4	0	Allen, Mr. Thomas Francis	male	30	1	2	01290	10.0175	00	0	100	female	Harland, 101/100
4	5	0	Allen, Mrs. Thomas Francis	female	20	1	2	01290	10.0175	00	0	100	female	Harland, 101/100
5	6	0	Anderson, Mr. Harry	male	44	0	0	00010	10.0175	00	0	3	female	New York, 101
6	7	0	Anderson, Mrs. Harry	female	40	0	0	00010	10.0175	00	0	10	female	Harland, 101
7	8	0	Anderson, Mr. Thomas	male	30	0	0	00010	10.0175	00	0	100	female	Harland, 101
8	9	0	Anderson, Mrs. Thomas	female	30	0	0	00010	10.0175	00	0	100	female	Harland, 101
9	10	0	Anderson, Mr. Thomas	male	30	0	0	00010	10.0175	00	0	100	female	Harland, 101

Twój własny projekt

- Po tych zajęciach powinieneś potrafić:
 - Uruchamiać aplikacje Python i Streamlit z VS Code / terminalu.
 - Budować proste interaktywne aplikacje z widgetami (suwaki, pola wejściowe, przyciski).
 - Wczytywać pliki CSV w Streamlit i wykonywać podstawowe EDA.
 - Używać paska bocznego, zakładek i kolumn do organizacji dashboardu.
 - Tworzyć tabele i wykresy (histogramy, wykresy słupkowe) bezpośrednio w aplikacji.
- Wszystko to będzie potrzebne w Twoim **projekcie indywidualnym**.
- Traktuj przykładowe aplikacje (app_2, app_3, app_4, app_5, app_6, app_7, app_8) jako **zestaw narzędzi**, który możesz kopiować, dostosowywać i rozszerzać.

Zadanie — Projekt indywidualny w Streamlit

Cel zadania

Zbuduj własną aplikację Streamlit do analizy wybranego zbioru danych.

1 Wybierz temat i dane

- Dowolna dziedzina: ekonomia, sport, zdrowie, środowisko.
- Dane z otwartych źródeł (Kaggle, GUS, Eurostat).
- Skonsultuj temat z prowadzącym.

2 Aplikacja musi zawierać:

- Wczytywanie pliku CSV przez `st.file_uploader`.
- Pasek boczny z filtrami i opcjami.
- Co najmniej **3 zakładki**: przegląd danych, statystyki, wykresy.
- Minimum **2 wykresy** (histogram, scatter lub bar).
- Obsługę brakujących danych.

3 Aplikacja powinna:

- Być łatwa w nawigacji.
- Prowadzić użytkownika od danych do wniosków.
- Zawierać tytuły i opisy na każdej zakładce.

Wskazówka

Zacznij od `app_8.py` jako szablonu i dostosuj do swojego tematu.

Praktyczne porady

- Zaczynij od przykładowych aplikacji z zajęć i **modyfikuj** je:
 - zastąp zestaw danych,
 - dostosuj zakładki i panele do swojego tematu,
 - dodaj własne wykresy i statystyki.
- Zachowaj prostotę, ale spójność:
 - jasna historia od danych do wniosków,
 - mniej dobrze wyjaśnionych wykresów jest lepsze niż wiele przypadkowych.
- Dokumentuj swoją pracę:
 - komentarze w kodzie,
 - tekst w aplikacji (tytuły, podpisy),
 - opcjonalny krótki raport pisemny.

Literatura i zasoby

- <https://streamlit.io/gallery>
- <https://docs.streamlit.io/develop/quick-reference/cheat-sheet>
- <https://github.com/PacktPublishing/Getting-started-with-Streamlit-for-Data-Science>
- <https://medium.com/data-science-collective/wait-this-was-built-in-streamlit-10-best-streamlit-design-tips-for-dashboards-2b0f50067622>